1) Postorden (A G F E B J I H C D): El último elemento es la raíz. (es decir, D).

Inorden (A B G E F D J H I C): La raíz divide al árbol. Todo lo que está a la izquierda de D es su subárbol izquierdo, y lo que está a la derecha es el derecho.

SubÁrbol derecho

Paso 1 → Buscar la raíz. Miramos el final del Postorden: La raíz es D.

Buscamos la D en el Inorden: [A B G E F] D [J H I C].

Conclusión: El subárbol izquierdo tiene los nodos {A, B, G, E, F} y el derecho tiene {J, H, I, C}.

Paso 2 →Analizar el subárbol derecho [J H I C]

En el Postorden, estos nodos aparecen al final: ... J I H C D. El último de este grupo (antes de la raíz D) es la C. Por lo tanto, C es el hijo derecho de D.

En el Inorden: Como todos los nodos (J, H, I) están a la izquierda de C en el Inorden, significa que C no tiene hijo derecho, y que J, H, I son todos descendientes por su rama izquierda.

Paso 3 → Analizar los descendientes de C. Ya sabemos que J, H, I están "debajo" de C. Vamos a ver cómo se organizan:

Miramos el Postorden para J, H, I: El orden es J I H .... El último es la H. Entonces, H es el hijo izquierdo de C.

Buscamos la H en el Inorden (dentro de su grupo J H I): [J] H [I]. Conclusión: H tiene un hijo izquierdo (J) y un hijo derecho (I).

SubÁrbol izquierdo

Paso 1 → Identificar raíz de grupo izquierdo.

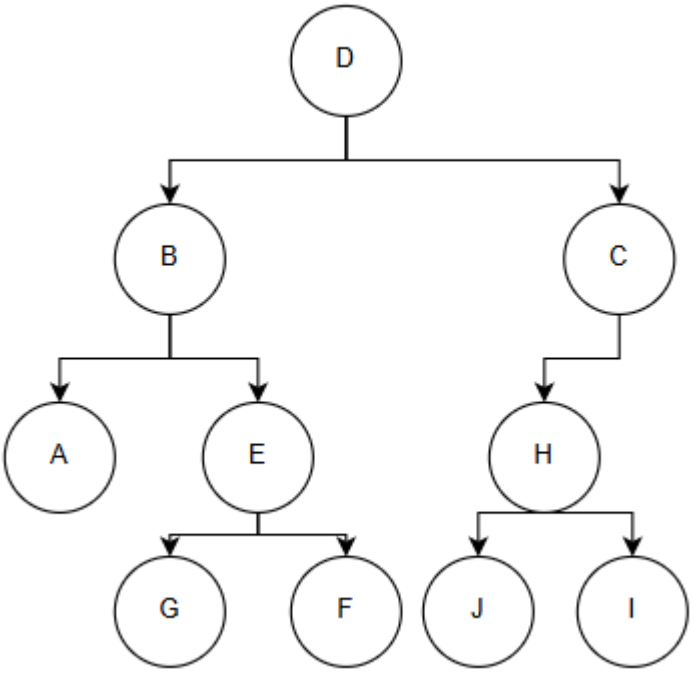
Miramos el Postorden original: A G F E B J I H C D. De nuestro grupo identificado [A B G E F]. El que aparece último es B en el postOrden, entonces B es el hijo izquierdo de D.

Paso 2 → Dividir el grupo usando B en el Inorden. A la izquierda de B está A (hijo izquierdo), y a su derecho G E F

Paso 3 → Organizar el subárbol derecho de B [G E F].

Miramos nuevamente el Postorden para ver cuál de estos tres aparece último: A G F E B.... El último de ellos es la E. Conclusión: E es el hijo derecho de B.

Ahora buscamos la E en el Inorden de ese grupito (G E F): [G] E [F]. A la izquierda de E está G. A la derecha de E está F.

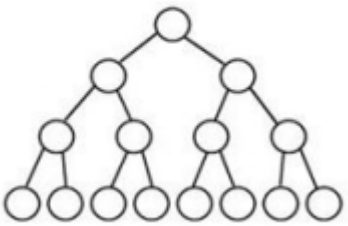


Rta = Los descendientes del nodo C son 3 [H, J, I].
Nota: El Postorden te da "el nodo padre" y el Inorden te dice "quién va a cada lado"

—----------

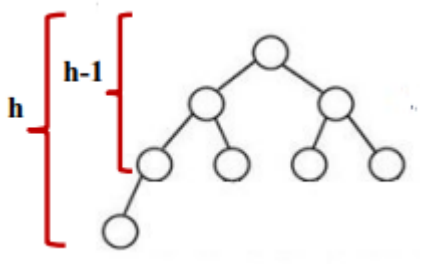
2) <u>Árbol Binario Lleno</u> = es un árbol de altura h donde cada nodo es de grado 2 (tiene 2 hijos) y todas las hojas se encuentran en el nivel h.

   La cantidad total de nodos en un árbol lleno de altura h se calcula como 2^{h+1} - 1, es decir, si tengo un árbol de altura 3, la cantidad total de nodos será 2^4-1 = 15.



   <u>Árbol Binario Completo</u> = es un árbol de altura h que es lleno hasta el nivel h-1, y cuyo nivel h (el último) se completa de izquierda a derecha.

   La cantidad total de nodos varía en el rango [2^h , 2^{h+1}-1], es decir, si tengo un árbol de altura 3, el rango será de [8, 15].



¿Todo árbol completo es lleno? Falso. Un árbol completo puede tener nodos con un solo hijo en el último nivel o tener hojas a diferentes alturas, siempre que se respete el llenado de izquierda a derecha, mientras que un árbol lleno exige que todos los nodos internos tengan dos hijos y las hojas estén a la misma altura.

¿Todo árbol lleno es completo? Verdadero. Por definición, un árbol completo de altura h debe ser lleno hasta el nivel h-1. Si el árbol es lleno hasta el nivel h, cumple con la condición de ser completo, ya que el último nivel está completamente ocupado.

—----------

3) Postorden → se procesan primero los hijos (izquierdo y derecho) y luego la raíz. Por lo tanto, el último nodo en postorden siempre es la raíz.

   Inorden → se procesa primero el hijo izquierdo, luego la raíz y luego el hijo derecho. Por lo tanto, el último nodo en Inorden es el que esté más a la derecha.

   La respuesta correcta es: **(b) El subárbol derecho de T es vacío**, ya que para que la raíz sea el último elemento procesado en el Inorden, no debe existir ningún hijo derecho o descendiente.

—----------

4) Análisis del algoritmo:

   Imprime dato (ocurre antes de imprimir a los hijos, como en un preorden)

   Llama recursivamente a los hijos izquierdos.

   Llama recursivamente a los hijos derechos.

   Imprime dato (después de visitar a todos los hijos, como en un postorden)

   Resultado:

   Imprime C (raíz)
   Llama al hijo izquierdo.
   Imprime A
   No tiene hijo izquierdo, llama al derecho.
   Imprime B
   Es hoja, no tiene hijos.
   Imprime B (cuando sale del nodo)
   Retorna al padre A
   Imprime A (cuando sale del nodo)

Retorna al padre C
Llama al hijo derecho E
Imprime E
Llama al hijo izquierdo D
Imprime D
Es hoja, entonces vuelve a imprimir D cuando sale del nodo.
Retorna al padre E
Llama al hijo derecho F
Imprime F
Es hoja, entonces vuelve a imprimir F
Retorna al padre E
Ya procesó sus hijos, entonces imprime E al salir
Retorna al padre C
Imprime C al salir.

Conclusión → **Imprime C A B B A E D D F F E C**

—----------

5) 6 5 * 7 3 − 4 8 + * +

La técnica para calcular una expresión postfija es: si es un operando se apila, si es un operador se desapilan los dos últimos valores. **El resultado final es 78**

Por ejemplo: llega el 6, se apila, llega el 5, se apila. Queda [6] [5], llega el operando *, se desapilan estos dos y se realiza el cálculo 6 * 5, el resultado (30) se apila. [30]

Llega el 7 y el 3, se apilan ambos. [30] [7] [3]

Llega el operando -, se desapilan el 7 y el 3 y se realiza el cálculo. Se apila el resultado (4). [30] [4]

Llega el 4 y el 8, se apilan. [30] [4] [4] [8]

Llega el +, se desapilan los últimos dos y se apila el resultado de 4+8. [30] [4] [12]

Llega el *, se desapilan los últimos dos y se apila el resultado (4*12 = 48). [30] [48]

Por último se realiza 30 + 48 = 78.

—----------

6) Respuesta correcta: **d) (((a – b) / c) + (d * e))**

—----------

7) Para que un árbol binario sea completo de altura h, debe ser lleno hasta la altura h-1 y tener al menos un nodo en el nivel h.

La fórmula para calcular la cantidad mínima de nodos que debe tener un árbol binario completo de altura 4 es: **2^4 = 16 (respuesta e)**

Nota = si el árbol fuera lleno, la fórmula sería 2^{h+1} - 1. Es decir, para altura 4, un árbol lleno tendría 2^{4+1} - 1 = 32 - 1 = 31 nodos.

8) Para construir el árbol a partir de una expresión postfija, la cátedra propone un algoritmo basado en el uso de una Pila.

Si leo un operando, creo un nodo con ese número y lo apilo.

Si leo un operador: creo un nuevo nodo con ese operador (será raíz)

desapilo el primer operando y lo pongo como hijo derecho

desapilo el segundo operando y lo pongo como hijo izquierdo

apilo el nuevo nodo (operador + hijos)

6 5 * 7 3 - 4 8 * + +

6, 5: Apilo nodos hoja con el 6 y el 5.

*: Desapilo el 5 (derecha) y el 6 (izquierda). Creo el subárbol (6 * 5) y lo apilo.

7, 3: Apilo nodos hoja con el 7 y el 3.

-: Desapilo el 3 (derecha) y el 7 (izquierda). Creo el subárbol (7 - 3) y lo apilo.

Estado de la pila: [(6*5), (7-3)]

4,8: Apilo nodos hoja con el 4 y el 8.

*: Desapilo el 8 (derecha) y el 4 (izquierda). Creo subárbol (4 * 8) y lo apilo.

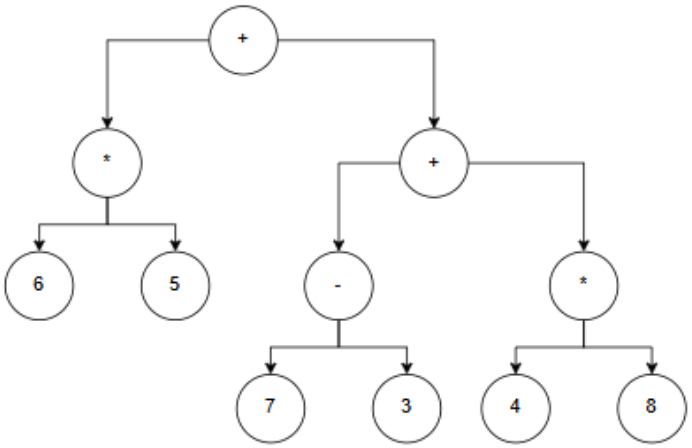
Estado de la pila: [(6*5), (7-3), (4*8)]

+: Desapilo el (4*8) (derecha) y el (7-3) (izquierda). Creo subárbol que sume esos dos resultados ((7-3) + (4*8)) y lo apilo.

Estado de la pila: [(6*5), ((7-3) + (4*8))]

+: Será la raíz final del árbol. Desapilo el bloque grande (derecha) y (6*5) (izquierda). Los uno con un +.

El árbol final queda:



9) Para construir el árbol a partir de una expresión infija, se debe convertir la expresión infija en postfija y a partir de esta construir el árbol.

Algoritmo para construir el árbol

- si es un operando, se coloca en la salida.
- si es un operador, se maneja una pila según la prioridad del operador en relación al tope de la pila.

  Si el operador tiene mayor prioridad que el tope de la pila → se apila
  Si el operador tiene menor/igual prioridad que el tope de la pila → se desapila colocándolo en la salida y se vuelve a comparar el operador con el tope de la pila.

- si es un (se apila
- si es un) se desapila todo hasta encontrar el paréntesis de inicio (
- si se llega al final de la expresión, se desapilan todos los elementos hasta que la pila quede vacía.

PRIORIDAD DE OPERADORES (de mayor a menor)

^ potencia
* / multiplicación y división
+ - suma y resta
Los '(' siempre se apilan y se desapilan solo cuando aparece el ')'.

EXPRESIÓN (A + (B * C)) * (D – E)

Pila → (, +, (, *
Salida → A B C

Entra el paréntesis de cierre), entonces desapilo todo hasta encontrar paréntesis de inicio. La pila queda (, +
Desapilé (*) → en la salida solo va el *, los paréntesis se descartan.

Pila → (, +
Salida → A B C *

Entra otro paréntesis de cierre. Saco todo
Pila →
Salida → A B C * +

Entra *, se apila
Entra (, se apila
Entra operando D, va a la salida
Pila → *, (
Salida → A B C * + D

Entra operador –, como el tope de la pila es un paréntesis, lo apilo
Entra E, va a la salida
Pila → *, (, -

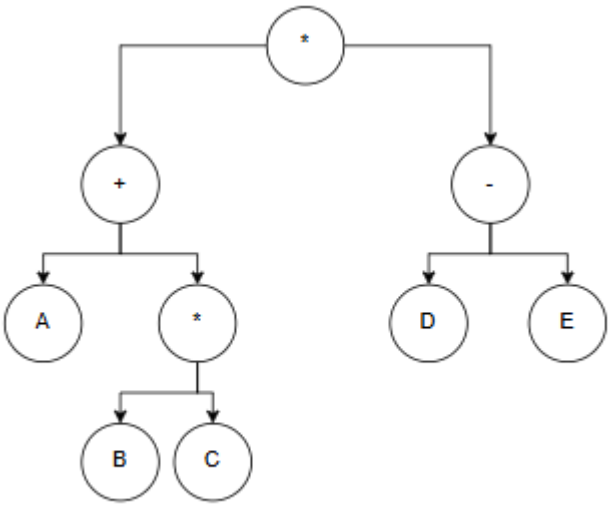
Salida → A B C * + D E

Entra paréntesis de cierre), desapilo (, - y mando a la salida el -
Pila → *
Salida → A B C * + D E -

Como llegué al final de la expresión, dejo la pila vacía. Desapilo el * que queda

Salida (Expresión postfija final) → A B C * D E - *

Algoritmo para construir árbol:
A, B, C: apilo como nodos hoja → [A] [B] [C]
*: Desapilo C (derecha) y B (izquierda), creo raíz * → [A] [(B * C)]
+: Desapilo el nodo que cree recién y A (izquierda) → [(A + (B * C))]
D, E: apilo como nodos hoja → [(A + (B * C))] [D] [E]
-: Desapilo E (derecha) y D (izquierda) → [(A + (B * C))] [(D - E)]
*: Desapilo los dos nodos → [(A + (B * C) * (D - E)]



—----------

10) Para construir el árbol a partir de una expresión prefija, se debe hacer:

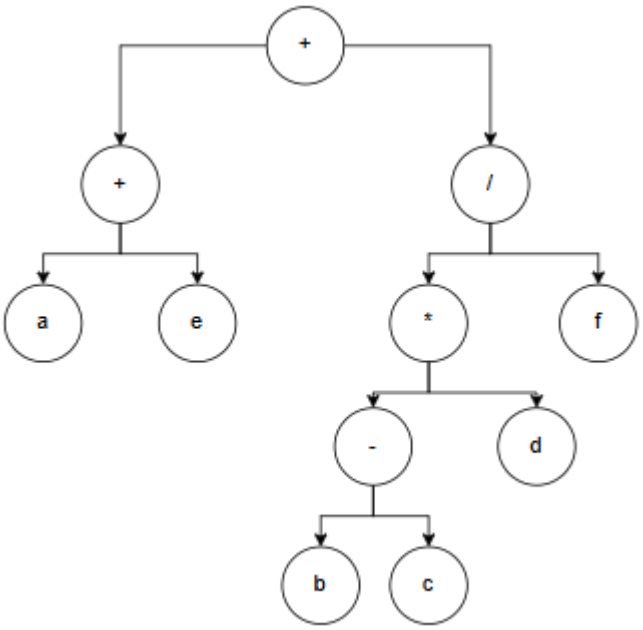
- si encuentro un operador → ese operador va a ser la raíz. Inmediatamente, abro dos caminos: uno para su hijo izquierdo y otro para su hijo derecho.

- si encuentro un operando → creo una hoja. Como las hojas no tienen hijos, ya no hay que buscar más. Vuelve arriba para terminar lo que falta. Es un proceso recursivo.

+ + a e / * - b c d f

+: raíz principal.
+: hijo izquierdo de la raíz principal. Como es otro operador, será el padre de un nuevo subárbol a la izquierda.
a: hijo izquierdo de +. Como es hoja, vuelvo arriba.

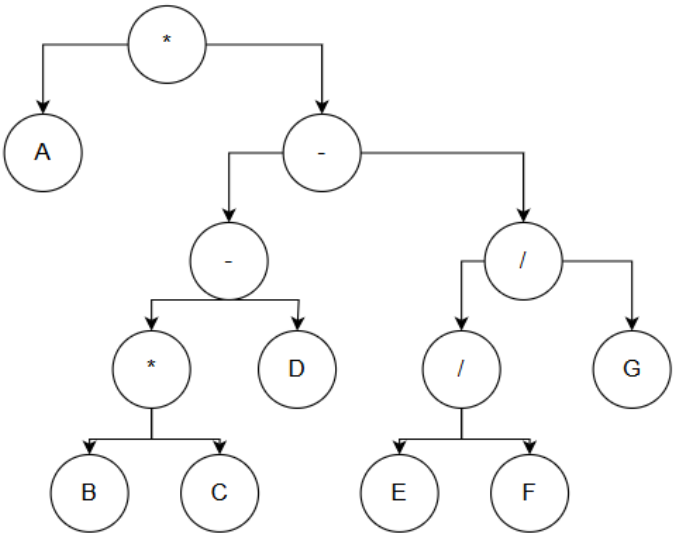
e: hijo derecho de +. Como es hoja, vuelvo arriba.
/: hijo derecho de la raíz principal. Será padre de un nuevo subárbol a la izquierda.
*: hijo izquierdo de /. Como es operador, tiene hijos.
-: hijo izquierdo de *. Es operador, tiene hijos.
b: hijo izquierdo de -. Vuelvo arriba porque es hoja.
c: hijo derecho de -. Vuelvo arriba porque es hoja.
d: hijo derecho de *. Es hoja, vuelvo arriba.
f: hijo derecho de /.



—----------

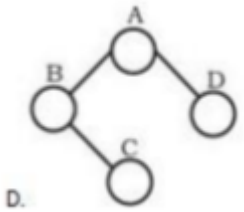
11) A B C * D - E F / G / - *

A, B, C: apilo como nodos hoja → [A] [B] [C]
*: desapilo C (derecho) y B (izquierdo). Creo raíz → [A] [B * C]
D: apilo → [A] [B * C] [D]
-: desapilo D (derecho) y [B * C] (izquierdo) → [A] [((B * C) - D)]
E, F: apilo → [A] [((B * C) - D)] [E] [F]
/: desapilo F(derecho) y E(izquierdo) → [A] [((B * C) - D)] [E / F]
G: apilo → [A] [((B * C) - D)] [E / F] [G]
/: desapilo G (derecho) y [E/F] (izquierdo) → [A] [((B * C) - D)] [((E/F) / G)]
-: desapilo los últimos dos → [A] [(((B * C) - D) - ((E/F) / G)))]
*: Raíz principal → [A * (((B * C)-D) - (E/F)/G)))]



—----------

12) La respuesta es el d.



Inorden procesa hijo izquierdo, raíz, hijo derecho. Entonces:

- Procesa subárbol B (que actúa como raíz), como no tiene hijo izquierdo, procesa el derecho (C) y sube
- Como ya procesó B, procesa A (raíz), y luego D que es el hijo derecho.

Se tiene un recorrido **BCAD**

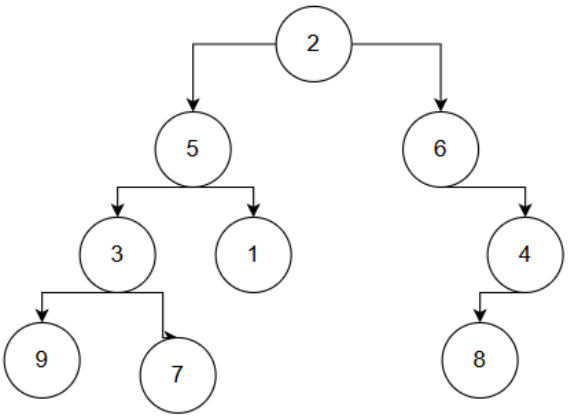
Preorden procesa raíz, hijo izquierdo, hijo derecho. Entonces:

- Procesa raíz A, va a su subárbol izquierdo.
- Procesa subárbol B (raíz), como no tiene hijo izquierdo, procesa el derecho (C) y sube
- Procesa el hijo derecho de A, que es D.

Se tiene un recorrido **ABCD**

—----------

13) Árbol con recorrido <u>preorden</u> es 2 5 3 9 7 1 6 4 8 y su recorrido <u>inorden</u> es 9 3 7 5 1 2 6 8 4.



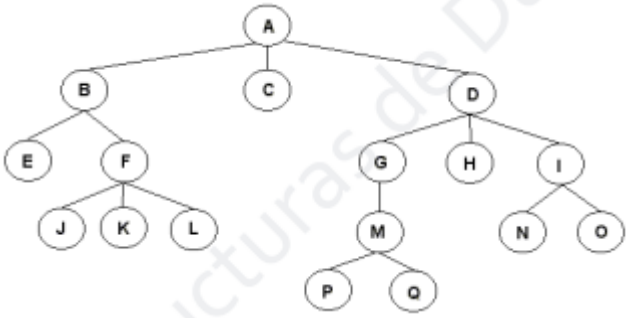
—----------

14) Si un árbol binario lleno tiene L hojas, el número total de nodos N siempre será el doble de la cantidad de hojas - 1. Es decir, la respuesta d) N = 2 * L - 1.

Ejemplo: Si mi árbol es de altura h = 2:

- Cantidad de hojas: $L = 2^2 = 4$.
- Total de nodos: N = (2 * 4) -1 = 7.
- Verificando con la fórmula de altura: $2^{2+1} - 1 = 8-1 = 7$.

—----------

15) 

i. A es la raíz del árbol.

ii. A es padre de B, C y D.

iii. E y F son hermanos, puesto que ambos son hijos de B.

iv. E, J, K, L, C, P, Q, H, N y O son las hojas del árbol.

v. El camino desde A a J es único, lo conforman los nodos A, B, F, J y es de largo 3

vi. G es ancestro de P, y por lo tanto G es descendiente de D.

vii. L no es descendiente de C, puesto que no existe un camino desde C a L.

viii. La profundidad/nivel de C es 1, de F es 2 y de P es 4.

ix. La altura de C es 0, de F es 1 y de D es 3

x. La altura del árbol es 4 (largo del camino entre la raíz y la última hoja)

Nota: La altura se mide como el camino más largo desde el nodo hasta una hoja.

i. Recorridos en profundidad

a) preorden → A B E F J K L C D G M P Q H I N O

b) inorden → E B J F K L A C P M Q G D H N I O

c) postorden → E J K L F B C P Q M G H N O I D A

ii. Recorrido por niveles → A B C D E F G H I J K L M N O P Q

—----------

16) Un árbol de grado k y altura h es completo si es lleno de altura h-1 y el nivel h se completa de izquierda a derecha. Esto significa que el número de nodos N variará dependiendo de cuantos nodos tenga el último nivel.

    El máximo de nodos ocurre cuando el árbol es lleno (todas las hojas están en el nivel h y todos los nodos internos tienen grado k). La fórmula es:

$$N_{max} = \frac{k^{h+1} - 1}{k - 1}$$

    El mínimo de nodos ocurre cuando el árbol es lleno hasta el nivel h-1 y tiene por lo menos un nodo en el nivel h. La fórmula es:

$$N_{min} = \frac{k^{h} + k - 2}{k - 1}$$

    Por ejemplo, si el árbol tiene altura 4 (camino más largo desde la raíz hasta una hoja) y grado 3 (cada nodo interno puede tener hasta 3 hijos):

    Máximo de nodos (árbol lleno) = $N_{max} = \frac{3^{4+1}-1}{3-1}$ = 121 nodos.

    Mínimo de nodos (árbol completo) = $N_{min} = \frac{3^{4}+3-2}{3-1}$ = 41 nodos.

—----------

17) El recorrido inorden en un árbol general visita: d) Primero el subárbol hijo más izquierdo, luego la raíz y luego los restantes subárboles hijos.

—----------

18) En un árbol general, la profundidad de un nodo n1 es: a) La longitud del único camino que existe entre la raíz y el nodo n1.

—----------

19) Un árbol general lleno de grado 4, tiene 21 nodos.

a) ¿Cuál es la altura del árbol? La altura del árbol es 2.

b) Desarrolle el proceso realizado para obtener la respuesta anterior.

Para llegar al resultado, debemos despejar la variable de la altura (h) de la fórmula de cantidad de nodos (N) de un árbol lleno de grado k:

Cantidad de nodos = 21
Grado del árbol = 4

La fórmula es:

$$N = \frac{k^{h+1} - 1}{k - 1}$$

Si reemplazamos:

$$21 = \frac{4^{h+1} - 1}{4 - 1}$$

Y despejo para encontrar h:

Paso el 3 (del 4-1) multiplicando a la izquierda = 21 * 3 = 4^{h+1} - 1

Y paso el 1 sumando = 63 + 1 = 4^{h+1}

Y busco un valor para la potencia, tal que al elevar el 4 me de 64 = 4^3. Por lo tanto:

$$4^3 = 4^{h+1}$$

$$3 = h + 1$$

$$h = 3 - 1$$

entonces, h = 2.

—----------

20) Árbol de grado 3 (k) y altura 4 (h). Usando la fórmula:

$$N_{min} = \frac{k^h + k - 2}{k - 1}$$

Y reemplazando k y h me queda que:

N min = 3^4 + 3 - 2 / 3-1
N min = 81 + 1 / 2
N min = 82 / 2 = 41 nodos. (respuesta b))